Machine Learning Basics Cheat Sheet

Module 9 · scikit-learn · ShopKart examples (`shopkart_clean.csv`, `random_state=42`)

Types of ML

Type	Predicts	Example	sklearn
Regression	A number	Order revenue (₹)	`LinearRegression`
Classification	A category	Returned? (1/0)	`DecisionTreeClassifier`, `LogisticRegression`
Clustering (unsupervised)	Groups, no target	Customer segments	`KMeans`

The workflow (copy this for every project)

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("shopkart_clean.csv")

# 1-2. Question, target y and features X
y = (df["returned"] == "Y").astype(int)                  # text -> 1/0
features = ["category", "city", "payment_mode", "delivery_days",
            "discount", "membership", "age", "unit_price"]
X = pd.get_dummies(df[features], drop_first=True, dtype=int)   # text
-> 0/1 columns

# 3. Split: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)    # stratify
for classification

# 4-5. Train and predict
model.fit(X_train, y_train)
pred = model.predict(X_test)

# 6. Evaluate (see below)
```

Models

```
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

reg  = LinearRegression()
tree = DecisionTreeClassifier(max_depth=4, random_state=42)
log  = make_pipeline(StandardScaler(), LogisticRegression())   # scale
```

```
first

proba = log.predict_proba(X_test)[:, 1]      # probability of class 1
(returned)
custom = (proba >= 0.2).astype(int)          # your own threshold
reg.coef_, reg.intercept_                    # what the regression
learned
tree.feature_importances_                    # what the tree found
useful
```

Regression metrics (in ₹)

Metric	Meaning	Good when	ShopKart revenue model
MAE	Average size of the miss	Low	₹2,403 (baseline ₹2,438)
RMSE	Like MAE, punishes big misses	Low	₹4,494 (baseline ₹4,909)
R^2	Share of variation explained (0–1)	High	0.16 (modest, realistic)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
mae = mean_absolute_error(y_test, pred)
rmse = mean_squared_error(y_test, pred) ** 0.5
r2 = r2_score(y_test, pred)
```

Always compare with a baseline (always predict the training average).

Classification metrics

	Predicted 0	Predicted 1
Actual 0	TN	FP (false alarm)
Actual 1	FN (missed)	TP (caught)

Metric	Formula	Question it answers
Accuracy	(TP + TN) ÷ all	How often right overall? (misleading if one class is rare)
Precision	TP ÷ (TP + FP)	When I say "yes", am I right?
Recall	TP ÷ (TP + FN)	Of all real "yes", how many did I catch?
F1	2·P·R ÷ (P + R)	Balance of precision and recall

```
from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
```

ShopKart returns (1,000 test orders, 137 returns):

- An always-"no" model has 86.3% accuracy and catches 0 returns.
- Logistic regression, by threshold:

Threshold	Caught	Precision	Recall
0.5	0	0.00	0.00
0.2	47	0.26	0.34
0.15	71	0.22	0.52

Lower threshold: more recall, less precision. Choose by business cost.

Overfitting

	Train score	Test score	Fix
Underfitting	Low	Low	More features, more flexible model
Good fit	Good	Close to train	–
Overfitting	Very high	Much lower	Limit `max_depth`, more data, cross-validation

ShopKart tree: depth 1 has train 0.862 / test 0.863. Depth 15 has train 0.941 / test 0.798. No limit has train 1.00 / test 0.76.

Data leakage checklist

Before using a feature, ask: **"Would I know this at the moment of prediction?"**

Leakage example	Why it leaks
`rating` to predict returns	Given after delivery or return. Accuracy jumps to 90.2%, but it cannot be used.
`unit_price` + `quantity` + `discount` to predict revenue	The formula itself (R^2 0.80 in disguise)
Using test data during training, scaling or choosing settings	The test score is no longer honest

K-Means in 5 lines

```
from sklearn.cluster import KMeans
cust = df.groupby("customer_id").agg(orders=("order_id", "count"),
        total_spend=("revenue", "sum"), avg_discount=("discount",
"mean"))
scaled = StandardScaler().fit_transform(cust)
cust["segment"] = KMeans(n_clusters=3, random_state=42,
n_init=10).fit_predict(scaled)
```

Common errors

Error	Fix
`could not convert string to float: 'Fashion'`	`pd.get_dummies` on the text columns
`Input contains NaN`	Fill or drop missing values first (e.g. rating)
`ConvergenceWarning` (logistic)	Add `StandardScaler` in a pipeline, or `max_iter=1000`

Error	Fix
Different numbers from the video	Set `random_state=42` everywhere